Lab 14 — Network Segmentation with Linux Namespaces

You will build three security zones (DMZ, internal, management) using network namespaces, then enforce a zone-to-zone policy with `iptables`.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free): - `iproute2`, `iptables` (pre-installed)

Step 1 — Create three zones

```
ip netns add dmz
ip netns add internal
ip netns add mgmt
ip netns list
```

Each namespace has its own routing table, interfaces, and firewall — exactly the property a real DMZ provides.

Step 2 — Wire each zone to a central router (the host)

```
for z in dmz internal mgmt; do
  ip link add veth-$z type veth peer name veth-${z}-r
  ip link set veth-$z netns $z
done

ip link set veth-dmz-r up
ip link set veth-internal-r up
ip link set veth-mgmt-r up
```

Assign IPs:

```
ip netns exec dmz      ip addr add 10.0.10.2/24 dev veth-dmz
ip netns exec internal ip addr add 10.0.20.2/24 dev veth-internal
```

```
ip netns exec mgmt     ip addr add 10.0.30.2/24 dev veth-mgmt
ip addr add 10.0.10.1/24 dev veth-dmz-r
ip addr add 10.0.20.1/24 dev veth-internal-r
ip addr add 10.0.30.1/24 dev veth-mgmt-r
for z in dmz internal mgmt; do
  ip netns exec $z ip link set veth-$z up
  ip netns exec $z ip link set lo up
done
```

```
ip netns exec dmz      ip route add default via 10.0.10.1
ip netns exec internal ip route add default via 10.0.20.1
ip netns exec mgmt     ip route add default via 10.0.30.1
sysctl -w net.ipv4.ip_forward=1
```

Step 3 — Verify everything can reach everything (no policy yet)

```
ip netns exec dmz ping -c1 10.0.20.2
ip netns exec internal ping -c1 10.0.30.2
```

Step 4 — Apply zoning policy

Rules: - DMZ → Internal: **deny** (DMZ is hostile) - Internal → DMZ: **allow** (clients reach published services) - Mgmt → Anywhere: **allow** (admin jump host) - Anywhere → Mgmt: **deny** (management is private)

```
iptables -P FORWARD DROP
```

```
iptables -A FORWARD -s 10.0.30.0/24 -j ACCEPT

iptables -A FORWARD -d 10.0.30.0/24 -j DROP

iptables -A FORWARD -s 10.0.20.0/24 -d 10.0.10.0/24 -j ACCEPT

iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

iptables -L FORWARD -n -v
```

Step 5 — Re-test the policy

```
echo "DMZ->Internal (should fail):";     ip netns exec dmz ping -c1 -W1 10.0.20.2 || true

echo "Internal->DMZ (should pass):";     ip netns exec internal ping -c1 -W1 10.0.10.2

echo "Mgmt->DMZ (should pass):";         ip netns exec mgmt ping -c1 -W1 10.0.10.2

echo "DMZ->Mgmt (should fail):";         ip netns exec dmz ping -c1 -W1 10.0.30.2 || true
```

This is exactly the "screened subnet" pattern in Security+ 3.2.

Free online tools

- **NIST SP 800-207 Zero Trust Architecture** — https://csrc.nist.gov/publications/detail/sp/800-207/final
- **Cloudflare Zero Trust learning** — https://www.cloudflare.com/learning/access-management/what-is-zero-trust/
- **draw.io / diagrams.net** — diagram zones: https://app.diagrams.net
- **AWS VPC reference architectures** — https://docs.aws.amazon.com/vpc/latest/userguide/

What you learned

- How namespaces deliver true layer-3 isolation on a single host.
- The default-deny posture (`-P FORWARD DROP`) and explicit allow rules.
- The screened-subnet / DMZ pattern in working code.